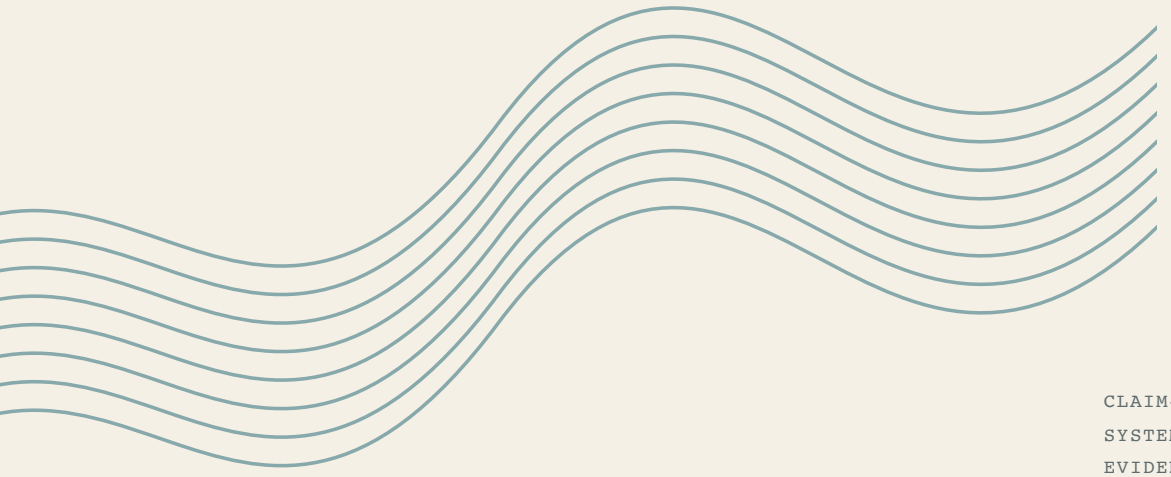
ENTER 뒤의 시스템

React 화면에서 generator turn, tool effect, recovery까지 한 요청의 생애를 따라간다

이 보고서는 구성요소를 나열하지 않는다. 사용자가 Enter를 누른 순간부터 화면과 실행 상태가 갈라지고 다시 합류하는 과정을 하나의 인과적 이야기로 읽는다.

제3자 복원 snapshot을 대상으로 하며, 관찰·외부 배경·프로젝트 추론을 분리한다.



CLAIM-LED PROTOTYPE V2
SYSTEMS MONOGRAPH × EVIDENCE NOTEBOOK
REPORT-ONLY CANDIDATE · 2026-09-21

BEFORE THE FIRST CHAPTER

이 보고서가 말할 수 있는 것과 말할 수 없는 것

독자가 알아야 할 첫 번째 사실은 기술이 아니라 근거의 경계다.

분석 대상은 배포 bundle에서 복원된 제3자 snapshot이다. 파일, 타입, 분기와 호출은 그 snapshot 안에서 관찰할 수 있지만, 그것만으로 upstream 개발자의 의도나 현재 공식 제품의 전체 동작을 확정할 수는 없다. 따라서 이 보고서는 보이는 구조를 출발점으로 삼되, "왜 그렇게 설계되었는가"를 말할 때는 반드시 추론임을 표시한다.

JavaScript generator와 React state처럼 언어·framework의 일반 동작은 공식 문서에서 가져온 외부 배경지식이다. 이 지식은 코드를 읽는 선행 개념을 제공하지만 프로젝트 내부의 특정 상태 전이를 스스로 증명하지 않는다. 반대로 local source의 함수나 타입은 현재 snapshot의 사실이지만 JavaScript와 React의 보편적 의미를 정의하지 않는다.

두 종류의 근거를 연결할 때 비로소 프로젝트 추론이 생긴다. 예를 들어 React의 state snapshot 개념과 local REPL의 event 소비 구조를 함께 보면, Enter handler와 agent turn 완료를 같은 시간 경계로 그려서는 안 된다는 편집 결론을 낼 수 있다. 이 결론은 유용하지만 관찰 사실과 동일한 확정성을 갖지는 않는다.

이 책은 그래서 두 종류의 지면을 사용한다. Systems Monograph 면에서는 한 요청의 인과관계를 연속 산문과 도해로 설명한다. Evidence Notebook 면에서는 그 설명을 멈추고 source, background, inference, unknown을 다시 분해한다. Notebook은 부록이 아니라 본문의 주장 강도를 조절하는 제동 장치다.

장 제목과 강조어를 지워도 본문만으로 논증이 남아야 한다.

- OBSERVED SOURCE FACT — 현재 복원 snapshot과 그로부터 생성된 local study에서 직접 확인한 내용
- EXTERNAL BACKGROUND — MDN, React, Anthropic 등 공식 자료가 설명하는 일반 개념
- PROJECT INFERENCE — 두 근거를 현재 프로젝트의 구조와 독자 여정에 연결한 해석

THE END-TO-END ARGUMENT

Enter는 명령 하나가 아니라 서로 다른 소유권을 통과하는 값이다

각 단계는 앞 단계가 만든 값을 받으며, 화면 갱신·model continuation·외부 effect·복구 판단은 같은 종류의 사건이 아니다.

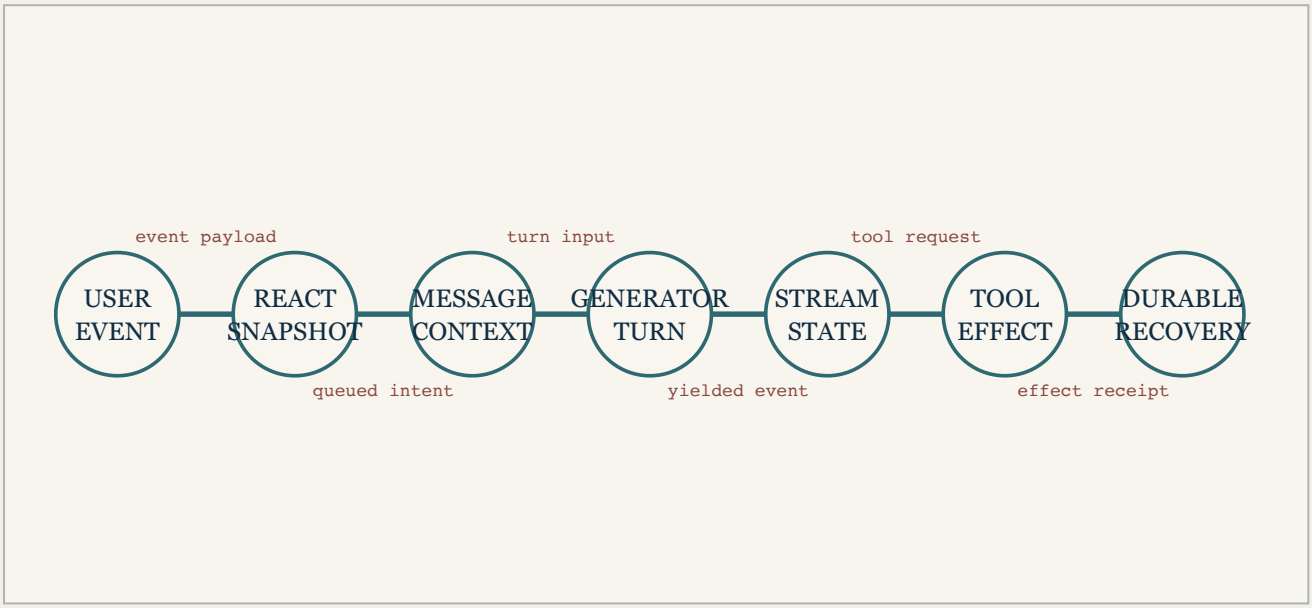


그림 1. 화살표는 "다음 기능"이 아니라 다음 owner에게 넘어가는 값을 표시한다. 이 값을 구분해야 재시도와 cleanup의 대상을 고를 수 있다.

처음 두 단계는 사용자 경험의 시간과 실행의 시간을 분리한다. Enter는 UI가 받은 사건이고, React update는 다음 render를 예약한다. 여기까지 진행됐다는 사실만으로 model 요청이 끝났거나 tool effect가 일어났다고 말할 수 없다.

중간 세 단계는 progressive observation을 만든다. 정규화된 message와 context가 generator turn의 입력이 되고, generator는 partial event를 내보내면서 실행 frame을 보존한다. stream consumer는 그 event를 화면에 투영하지만 canonical message가 닫히는 시점은 별도 protocol이 정한다.

마지막 두 단계는 외부 세계와 실패 이후를 다룬다. model이 tool을 요청해도 client의 validation과 permission을 지나기 전에는 effect가 아니다. effect가 끝난 뒤에는 result와 durable record가 남아야 다음 model call이나 recovery가 같은 일을 두 번 하지 않는다.

다음 장으로 넘기는 질문 Enter를 눌렀을 때 UI는 곧바로 "모델에게 보내기"를 실행하는가, 아니면 먼저 현재 mode와 입력 상태를 해석하는가?

READER QUESTION

Enter는 언제 “전송”이 되고, 언제 다른 행동이 되는가

키보드 사건의 이름은 같아도, UI mode와 guard가 다르면 다음 owner에게 넘기는 값이 달라진다.

사용자가 Enter를 누르면 가장 먼저 일어나는 일은 agent loop의 시작이 아니다. local study가 복원한 REPL 흐름에서는 selection 확인, suggestion 수락, agent routing, 빈 입력 처리처럼 현재 mode에 따라 Enter의 의미가 먼저 갈린다. 이 분기는 단순한 편의 기능이 아니라 키보드 사건을 application intent로 바꾸는 해석 경계다.

이 경계를 건너기 전에는 “사용자 메시지”가 아직 만들어지지 않았을 수 있다. 선택 목록이 열려 있다면 Enter는 선택을 확정하고, suggestion이 활성화되어 있다면 입력을 보완한다. 빈 입력은 전송을 만들지 않을 수 있으며, 별도의 agent route가 선택된 상태라면 같은 문자열도 다른 실행 경로로 전달될 수 있다.

따라서 보고서가 Enter 다음에 곧바로 model box를 배치하면 두 책임을 하나로 접게 된다. UI는 현재 interaction mode를 해석하고, application boundary는 그 결과를 message 또는 control action으로 정규화한다. agent turn은 그 뒤에 시작한다. 이 순서를 보존해야 중복 Enter, modal input, cancellation race를 어느 층에서 다룰지 설명할 수 있다.

OBSERVED
기존 source study는 Enter 처리에 mode별 guard가 존재하고, Enter가 항상 model 전송을 의미하지 않는다고 정리한다.

INFERENCE
따라서 PDF의 첫 화살표는 `Enter → Model`이 아니라 `Enter → UI intent`여야 한다.

Worked trace — 평범한 한 줄 입력

사용자가 “이 변경의 위험을 설명해줘”라고 입력하고 Enter를 누른다. selection과 suggestion이 없고 입력이 비어 있지 않다는 guard가 확인되면 UI는 문자열과 현재 interaction context를 제출 의도로 바꾼다. 아직 assistant output은 없고, pending UI는 turn 완료가 아니라 “입력이 다음 owner에게 넘어갔다”는 사실만 보여 준다.

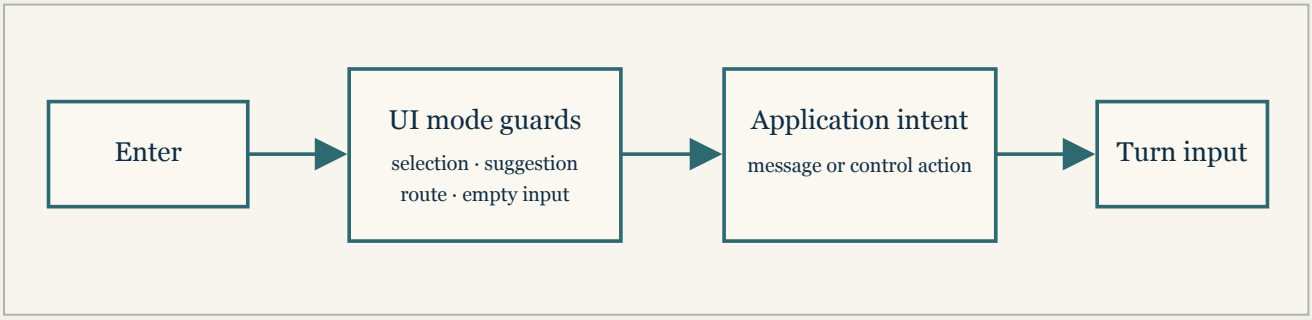


그림 2. 같은 key event라도 mode guard를 통과한 뒤에야 message 또는 control action이라는 의미를 얻는다.

다음 단계 application intent가 만들어져도 handler가 보고 있는 state와 다음 render의 state는 아직 같지 않다.

CLAIM UNDER REVIEW

“Enter handler가 끝났다는 사실은 agent loop가 끝났다는 뜻이 아니다.”

EXTERNAL BACKGROUND

React가 제공하는 시간 모델

React 공식 문서는 event handler, render별 state snapshot, handler가 끝난 뒤 처리되는 update queue를 구분한다. handler 안에서 setter를 호출해도 이미 실행 중인 handler가 보는 local state가 즉시 다른 값으로 바뀌는 것은 아니다. update는 다음 render를 요청한다.

```
function onEnter() {
  setPending(true)      // 다음 render를 위한 update
  submit(currentInput)  // 현재 render가 포착한 값 사용
}
```

이 코드는 React 개념을 설명하는 의사코드이며 local project source의 인용이 아니다.

OBSERVED SOURCE FACT

현재 snapshot에서 볼 수 있는 연결

기존 project study는 provider event가 query generator에서 yield되고, REPL consumer가 message와 state를 갱신하며, React/Ink가 그 state의 projection을 그린다고 정리한다. 이는 “API가 React를 직접 호출한다”는 모델과 다르다.

PROJECT INFERENCE

보고서에 적용하는 결론

sequence diagram은 최소한 UI event, queued render update, normalized message, generator event를 서로 다른 lane에 둬야 한다. pending indicator는 turn의 terminal state가 아니라 UI projection 상태로 설명한다.

경쟁 설명: “setPending(true)가 호출됐으니 turn이 시작됐다”고 단정할 수 있는가?

아니다. setter 호출은 UI update request를 설명할 뿐, message 정규화·model request·tool effect의 성공을 증명하지 않는다. local implementation이 두 동작을 같은 callback에서 연달아 호출하더라도 소유권과 실패 경계는 분리해 읽어야 한다.

SOURCES · React responding to events / state as a snapshot / queueing updates · local source study React/Ink chapter · CLM-002 / CLM-004

FROM INTERACTION TO EXECUTION

화면의 state는 그대로 model에 전달되는 payload가 아니다

turn은 입력 문자열뿐 아니라 현재 message history, tools, settings와 cancellation context를 함께 묶어 시작한다.

UI에서 제출 의도가 생긴 다음에는 그 의도를 실행 가능한 message와 context로 정규화해야 한다. 이 단계가 없으면 화면에서 보이는 문자열, React state에 저장된 배열, provider에게 보내는 canonical message가 같은 값이라고 오해하기 쉽다. 실제 시스템에서는 각 표현이 서로 다른 갱신 주기와 실패 위험을 가진다.

local study는 REPL이 render state만 소유하지 않고 message working set, query lifecycle, permission interaction까지 조정하는 복합 component라고 지적한다. 특히 messagesRef와 React state는 같은 대화의 서로 다른 시간 표현일 수 있다. ref는 synchronous control path가 최신 값을 보도록 돕고, React state는 다음 render에서 화면을 갱신한다.

turn context에는 현재 tools와 permission capability, model/settings, abort signal처럼 화면에 직접 보이지 않는 요소도 들어간다. 여기서 중요한 것은 항목 수가 아니라 provenance다. 어느 설정이 이번 turn에 들어왔고, 어떤 tool 목록과 permission mode가 선택됐는지를 기록해야 이후 행동 변화를 설명할 수 있다.

이 단계에서 생기는 대표적 실패는 stale context다. UI에는 최신 입력이 보이지만 turn이 이전 message 배열을 읽거나, 화면에는 활성 tool이 표시되지만 실제 context에는 갱신 전 registry가 들어갈 수 있다. 그러므로 보고서는 "state가 있다"가 아니라 누가 언제 snapshot을 만들고 어떤 digest나 identity로 다음 단계에 넘기는지를 설명해야 한다.

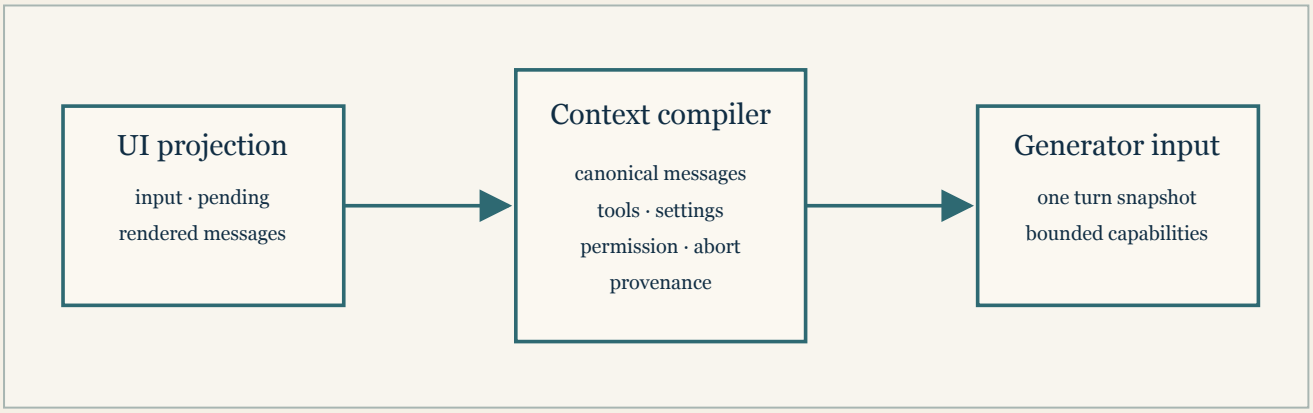


그림 3. context compiler는 화면을 복사하는 것이 아니라 실행에 필요한 canonical values와 capability를 한 turn snapshot으로 묶는다.

다음 단계 고정된 turn input을 받은 generator는 모든 일을 한 번에 반환하지 않고, 관찰 가능한 중간 event와 terminal을 다른 시점에 내보낸다.

LANGUAGE MECHANISM IN SERVICE OF THE SYSTEM

yield는 여러 번 return하는 문법이 아니다

중간 관찰을 내보내면서도 같은 실행 문맥을 보존한다는 점이 turn lifecycle의 핵심이다.

일반 함수의 return은 현재 호출 frame을 끝낸다. generator의 yield는 다르다. instruction pointer, local variable, try/finally 문맥을 보존한 채 iterator result를 호출자에게 돌려주고, 다음 resume를 기다린다. 소비자가 `next()`를 호출하면 새 함수를 시작하는 것이 아니라 같은 frame이 yield 다음 지점에서 계속된다.

이 차이는 agent turn에서 단순한 문법 취향이 아니다. model stream의 partial text, tool request, progress event를 UI나 SDK에 즉시 노출하면서도 turn 내부는 다음 provider event와 tool result를 기다려야 한다. generator는 "관찰 가능한 중간 상태"와 "계속 살아 있는 실행"을 동시에 표현한다.

```
async function* query() {
  yield { type: 'message_delta', text: '안' }
  yield { type: 'message_delta', text: '녕' }
  return { reason: 'end_turn' }
}
```

두 글자가 화면에 보였다는 사실은 terminal을 뜻하지 않는다. 각 yield는 partial observation이고, return 또는 surrounding protocol이 end turn을 확정한다. 오류는 `throw()` 경로로, 소비자의 조기 종료는 `return()` 경로로 frame 안에 전달될 수 있으며, finally는 cleanup을 실행할 기회를 제공한다.

BACKGROUND
MDN은 generator의 `next(value)`, `throw()`, `return()`과 yield suspension을 설명한다.

OBSERVED
기존 reader edition은 query event와 return을 다른 채널로 읽고, terminal reason을 별도 추적해야 한다고 정리한다.

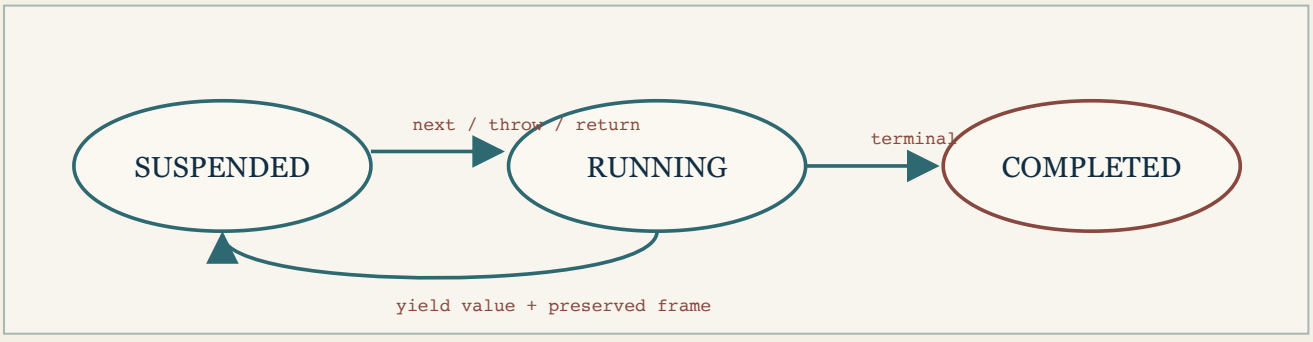


그림 4. yield는 completed로 가는 중간 return이 아니라 running과 suspended 사이를 오가는 protocol boundary다.

다음 단계 generator가 내보낸 event를 UI가 곧바로 그릴 수는 있지만, partial event를 canonical message로 정산하는 책임은 남아 있다.

WORKED PROTOCOL TRACE

“yielded value와 terminal result는 같은 채널로 소비된다고 가정하면 완료와 중간 상태를 혼동한다.”

T0 · CALL

`query()`는 iterator를 만든다

함수 body 전체가 끝난 것이 아니라, 소비자가 진행을 요청할 수 있는 실행 protocol이 만들어진다.

T1 · NEXT

첫 provider delta가 yield된다

소비자는 `{done:false}`인 event를 받아 화면 projection을 갱신한다. generator frame은 다음 event를 기다리며 보존된다.

T2 · RESUME

같은 frame이 다시 실행된다

새 query 호출이 아니다. local accumulator와 try/finally 문맥이 이어지며 다음 partial block이나 tool request를 처리한다.

T3 · TOOL

tool request를 yield해도 effect는 아직 일어나지 않았다

client는 request를 검증하고 permission을 확인한 뒤 실행 결과를 다음 continuation에 돌려준다.

T4 · RETURN

terminal reason이 turn을 닫는다

`done:true`의 value가 일반 `for await` body에 보이지 않을 수 있으므로 surrounding protocol이 terminal reason을 별도로 보존해야 한다.

T× · CANCEL

오류·취소는 cleanup 경로를 연다

`throw()` 또는 `return()`이 suspended frame으로 전달되며, finally에서 stream·resource cleanup을 수행할 수 있다.

EXTERNAL BACKGROUND

pause/resume와 iterator control은 MDN Generator/yield에서 확인한다.

OBSERVED SOURCE FACT

local reader edition은 yield 수가 아니라 event의 partial state와 canonical message가 닫히는 시점을 추적해야 한다고 명시한다.

LIMITATION · 복원 snapshot은 모든 provider adapter와 cleanup 경로의 build parity를 보장하지 않는다.

PROGRESSIVE OBSERVATION

화면에 보이기 시작한 응답은 아직 완성된 message가 아니다

stream consumer는 partial event를 즉시 투영하면서도 block identity와 terminal metadata를 보존해야 한다.

Streaming의 가치는 첫 token을 빨리 보여 주는 데만 있지 않다. provider가 text, thinking, tool use와 message metadata를 서로 다른 event로 보낼 때 consumer는 이 조각들이 어느 block과 message에 속하는지 복원해야 한다. UI는 아직 열린 message를 보여 줄 수 있지만 다음 model request는 정산된 canonical message를 필요로 한다.

local study는 일반 text/thinking event가 즉시 yield되면서 assistant message accumulator에도 쌓이고, tool use block은 별도 follow-up 요구를 만든다고 정리한다. recoverable error는 바로 terminal로 노출되지 않고 recovery 판단을 위해 잠시 보류될 수 있으며, fallback에서는 이전 partial array를 그대로 믿지 않고 tombstone이나 초기화로 관찰 상태를 고친다.

여기서 React의 역할은 provider event를 해석하는 것이 아니라 바뀐 application state를 terminal element tree로 투영하는 것이다. consumer가 replace와 append를 잘못 고르면 화면은 중복 text나 사라진 tool block을 보게 된다. 반대로 canonical accumulator가 잘못되면 화면이 멀쩡해도 다음 turn의 context가 손상될 수 있다.

따라서 성능 설명도 “React가 batching한다”에서 멈출 수 없다. chunk arrival, consumer state transition, render/commit, Ink layout, ANSI diff, stdout flush가 서로 다른 비용을 만든다. 정확한 timing은 scheduler와 renderer 버전에 따라 달라질 수 있으므로 이 prototype은 고정된 millisecond 수치를 주장하지 않는다.

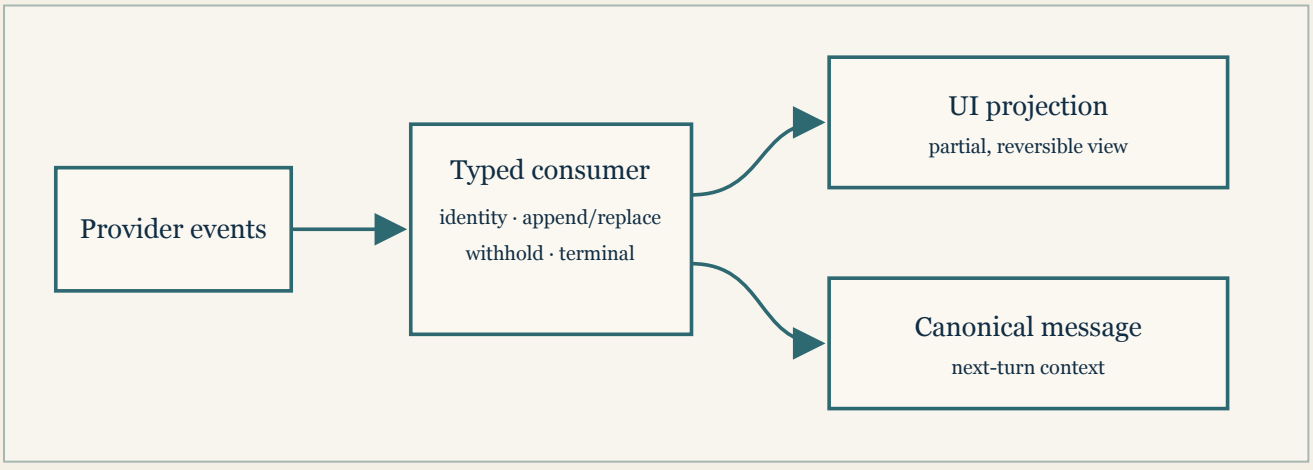


그림 5. 같은 stream event가 화면과 canonical message에 모두 영향을 주지만, 두 projection의 완료 조건은 다르다.

다음 단계 stream에서 tool request가 나타나면 model의 제안과 client가 실제로 일으키는 effect 사이에 새로운 경계가 열린다.

THE EFFECT BOUNDARY

model이 tool을 요청해도 외부 세계는 아직 바뀌지 않았다

validation, permission, hook, sandbox와 executor는 서로 다른 질문에 답하며 실패 의미도 다르다.

tool use는 model이 구조화된 JSON을 만들었다는 사실에서 시작한다. 그러나 name과 input이 존재한다고 곧바로 실행할 수 있는 것은 아니다. client는 등록된 tool인지 찾고, schema와 path 같은 입력을 정규화하며, permission rule과 human decision을 적용한다. pre-hook는 허용된 입력을 거부하거나 바꿀 수 있고, sandbox는 executor가 실제로 행사할 수 있는 capability를 제한한다.

이 단계들을 하나의 “allowed” boolean으로 접으면 실패를 설명할 수 없다. permission denial은 effect가 의도적으로 일어나지 않은 상태다. executor error는 effect를 시도했지만 완료하지 못한 상태일 수 있다. timeout은 외부 effect가 실제로 일어났는지 불명확한 상태를 만들 수 있다. retry policy는 이 차이를 알아야 같은 파일 쓰기나 command를 두 번 수행하지 않는다.

Anthropic의 client tool loop 설명도 model request와 client effect, tool result, continuation을 분리한다. model은 어떤 tool을 어떤 input으로 호출하고 싶은지 요청한다. client가 실행한 뒤에는 성공, 오류 또는 거부를 `tool_result`로 되돌리고, model은 그 결과를 다음 reasoning에 사용한다. 요청과 결과 사이가 application의 권한·감사·복구 경계다.

BACKGROUND
공식 tool-use 문서는 client-side effect와 `tool_result` continuation을 구분한다.

INFERENCE
PDF에서는 model lane과 effect lane 사이에 permission gate를 명시해야 한다.

Worked trace — 파일 읽기 요청

model이 경로와 함께 read tool을 요청한다. client는 tool identity와 schema를 확인하고 canonical path를 만든다. permission rule이 read를 허용하더라도 sandbox에서 해당 경로가 보이지 않으면 executor는 실패한다. 이때 기록해야 하는 것은 단순한 `false`가 아니라 어느 단계가 어떤 input을 보고 어떤 결과를 냈는지다.

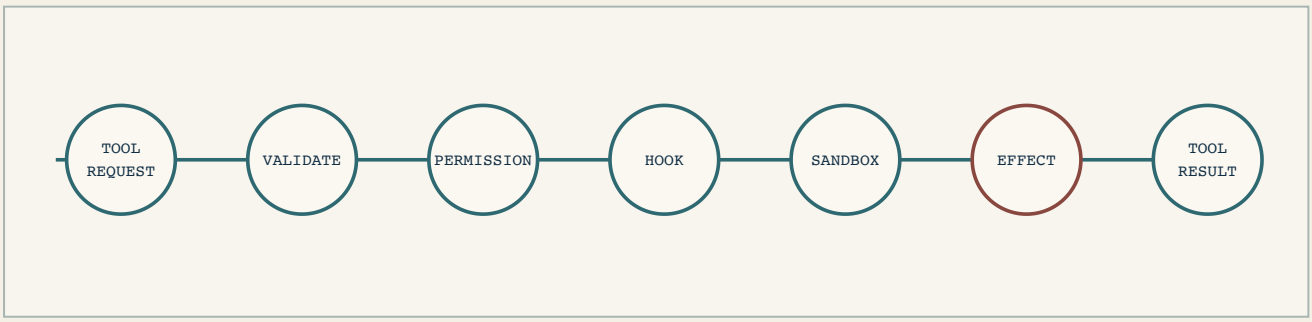


그림 6. 각 원은 기능이 아니라 서로 다른 결정·실패·재시도 경계다. effect만 외부 세계를 바꾼다.

다음 단계 result가 돌아와도 turn이 안전하게 끝났다는 뜻은 아니다. durable record가 무엇을 남겼는지 확인해야 복구할 수 있다.

COUNTEREXAMPLE-DRIVEN REVIEW

“tool 실패”라는 한 문장으로는 retry 여부를 결정할 수 없다.

관찰	실제 의미	model에 돌려줄 값	재시도 전 확인
Permission denied	effect가 의도적으로 시작되지 않았다.	거부 이유가 포함된 tool result 또는 control response	입력 변경이나 사용자 결정 없이는 같은 요청을 반복하지 않는다.
Executor error before commit	실행을 시도했지만 외부 상태 변경 전 실패했다.	구체적 실행 오류	commit boundary와 idempotency를 확인한다.
Timeout after request	effect 발생 여부가 불명확할 수 있다.	unknown/timeout 상태	receipt, external state 또는 recovery token을 조회한다.

▤ EXTERNAL BACKGROUND

client tool loop의 기본 분리

model request → client execution → tool result → continuation이라는 순서는 effect가 model 내부에서 자동으로 일어나지 않음을 보여 준다.

● OBSERVED SOURCE FACT

현재 local study가 보여 주는 더 세밀한 경계

lookup, parse, validation, permission, hook, call, normalize가 한 pipeline으로 정리되어 있으며, unknown tool도 model이 self-correction할 수 있도록 error result로 돌려주는 경로가 기술돼 있다.

◆ PROJECT INFERENCE

도해와 운영 설명에 반영하는 방식

permission과 effect를 하나의 방패 아이콘으로 표시하지 않는다. 각각의 입력, 결정, side effect와 receipt를 별도 node로 그리고, timeout은 실패가 아니라 “effect 여부 unknown”으로 표시한다.

Unknown: 복원 snapshot만으로 모든 외부 tool의 idempotency와 server-side commit을 확정할 수 있는가?

확정할 수 없다. client에서 관찰 가능한 요청·result와 local durability를 설명하되, server-side settlement는 별도 source가 없으면 unknown으로 남겨야 한다.

SOURCES · Anthropic tool-use flow · local source study tool pipeline · CLM-003 · reconstruction limitations retained

AFTER INTERRUPTION

복구는 다시 시작하는 일이 아니라 이미 일어난 것을 구분하는 일이다

화면, generator, effect, transcript는 서로 다른 속도로 사라지고 남는다.

terminal 화면이 닫혔다고 대화와 effect가 모두 사라지는 것은 아니다. 반대로 UI에 마지막 assistant text가 보인다고 canonical transcript와 tool result pairing이 모두 정산됐다고 말할 수도 없다. recovery는 먼저 어떤 layer가 durable하고 어떤 layer가 projection인지 분류하는 일에서 시작한다.

local study는 transcript write가 chain identity를 남기고, leaf에서 root로 conversation을 재구성하며, 병렬 tool result의 orphan을 복원하는 경로를 다룬다. compaction은 긴 history를 줄이지만, 압축 전후의 context와 pending summary가 올바르게 연결되지 않으면 다음 turn이 서로 다른 대화를 이어받을 수 있다.

generator cancellation도 별도다. consumer가 조기에 iteration을 끝내면 suspended frame에 return이나 abort가 전달되어야 한다. cleanup이 끝나기 전에 새 turn을 시작하면 같은 provider stream이나 tool resource가 중첩될 수 있다. UI의 pending flag를 false로 바꾸는 것만으로 이 자원을 닫을 수는 없다.

따라서 recovery decision은 최소한 마지막 canonical message, outstanding tool use/result pair, effect receipt, transcript checkpoint, cancellation state를 읽어야 한다. 이 증거가 충분하면 resume할 수 있고, effect는 남았지만 result가 없다면 compensate하거나 reconciliation을 해야 하며, 어느 쪽인지 모르면 자동 반복 대신 hold해야 한다.

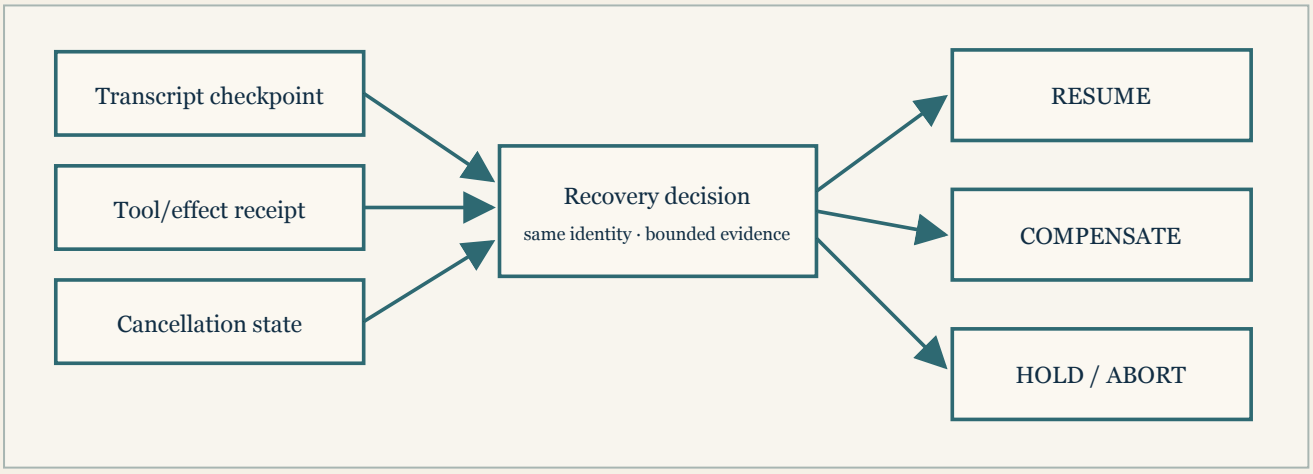


그림 7. recovery는 UI 상태가 아니라 durable evidence를 같은 identity로 합류시킨 뒤 resume, compensate, hold를 고르는 과정이다.

장 결론 한 turn의 완료는 화면의 마지막 frame이 아니라 message, effect, transcript와 cleanup이 서로 모순 없이 정산된 상태다.

CASE RECONSTRUCTION

사용자가 Enter를 누른 순간부터 복구 가능한 기록이 남을 때까지, 같은 "진행 중"이라는 말은 단계마다 다른 뜻을 가진다.

01 · INTENT

UI는 key event를 application intent로 바꾼다

selection·suggestion·route guard가 먼저 평가된다. 결과는 message 또는 control action이다.

02 · SNAPSHOT

React update는 다음 render를 예약한다

handler가 보는 snapshot과 turn terminal은 동일하지 않다.

03 · CONTEXT

message, tools, settings, permission과 abort가 한 turn input으로 묶인다

provenance가 없으면 stale context와 capability drift를 설명할 수 없다.

04 · YIELD

generator는 frame을 보존한 채 partial event를 내보낸다

보이는 text는 아직 terminal result가 아니다.

05 · EFFECT

tool request는 client의 permission과 execution을 거쳐야 한다

request, denial, executor error, timeout은 다른 recovery 의미를 가진다.

06 · DURABLE

transcript와 receipt가 다음 결정을 위한 사실을 남긴다

중단 뒤에는 이 사실을 읽고 resume, compensate, hold를 선택한다.

◆ EDITORIAL DISPOSITION

최종 보고서는 이 trace를 본문 spine으로 유지한다. component별 특징 표는 appendix로 보내고, 각 장은 앞 단계의 output value에서 시작해 다음 단계의 input value로 끝낸다.

검증자가 물어야 할 마지막 질문: 이 설명은 어느 지점에서 source보다 더 강한 확신을 말하고 있는가?

WHAT THE READER SHOULD CARRY FORWARD

좋은 agent runtime 설명은 “무엇이 있다”보다 “언제 바뀌는가”를 말한다

React, generator, tool과 transcript는 각각 다른 시간을 표현하며, 시스템은 그 시간들을 무리하게 하나로 만들지 않을 때 복구 가능해진다.

이 보고서의 출발점은 두 개의 구현 직관이었다. 하나는 generator가 progressive observation과 suspension을 표현한다는 것, 다른 하나는 React/Ink가 application state를 terminal 화면으로 투영한다는 것이다. 그러나 두 직관만으로는 permission, effect, durability와 recovery를 설명할 수 없다.

전체 흐름을 따라가면 각 기술의 위치가 선명해진다. React는 handler와 render snapshot의 시간을 관리한다. context compiler는 현재 turn이 사용할 canonical inputs와 capabilities를 묶는다. generator는 partial event와 terminal을 분리한다. client tool loop는 model request를 permission과 effect로 연결한다. transcript는 중단 후에도 남는 사실을 제공한다.

이 구조를 책으로 표현할 때도 같은 원칙이 필요하다. Systems Monograph 면은 원인과 결과를 끊지 않고 설명한다. Evidence Notebook 면은 그 설명을 다시 source와 limitation으로 분해한다. 둘 중 하나만 있으면 각각 근거 없는 유창함이나 맥락 없는 증거 목록이 된다.

따라서 이후의 본편 작성은 더 많은 카드나 키워드를 추가하는 방향이 아니다. 한 장마다 하나의 material claim을 선택하고, 독자의 질문에서 시작해 worked trace와 failure path를 거친 뒤 다음 장으로 넘길 값을 명시하는 방향이어야 한다. 시각 디자인은 이 논증을 읽을 수 있게 만드는 편집 장치로 남는다.

$$\text{UI event} \neq \text{render update} \neq \text{generator terminal} \neq \text{external effect} \neq \text{durable settlement}$$

이 식은 구현의 모든 세부를 대신하지 않는다. 다만 서로 다른 경계를 한 사건으로 접지 않기 위한 독서 규칙을 제공한다.

REFERENCE NOTE

근거를 다시 찾을 수 있게 남긴다

▤ OFFICIAL BACKGROUND

MDN Generator — generator object의 pause/resume와 iterator control.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

MDN yield — yield suspension과 resume value.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/yield

React — responding to events, state as a snapshot, queueing state updates.

https://react.dev/learn/responding-to-events · /state-as-a-snapshot · /queueing-a-series-of-state-updates

Anthropic — model request, client tool execution, tool result, continuation.

https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works

● PROJECT-LOCAL EVIDENCE

Claude Code source study, Architecture & Algorithms reader edition, language mechanisms, book plan과 editorial rationale를 사용했다.

`.collab/artifacts/claude-code-source-study-2026-09-20/**`
`.collab/artifacts/claude-code-architecture-algorithms-study-2026-09-20/book-edition-2026-09-21/**`

◆ LIMITATIONS

분석 대상은 제3자 복원 snapshot이다. 누락 모듈과 build parity 한계가 있으며 upstream intent, 현재 공식 제품 동작, server-side settlement를 확정하지 않는다. 두 영상은 출발 렌즈로만 사용했고 transcript를 독립 검증하지 않았다.

상태: report-only candidate. 독립 Fact/Visual validation, acceptance, publication, settlement는 수행되지 않았다.